

Technical Design Document – Memory Card Game

Repository: [thedeepak12/memory-card](#)

Tech Stack: React • Vite • Tailwind CSS

Purpose

This document defines the architecture, data models, control flows, component contracts, and quality considerations for the Memory Card Game. Mermaid diagrams are included as fenced code blocks so they render in GitHub and Mermaid-compatible tools.

Project Overview (from brief)

A browser-based memory matching game built with React and Tailwind CSS. Users flip cards to find matching pairs; the app tracks current and best scores, handles errors gracefully, and displays a win modal upon completion.

Section	Description
1. System Overview	Goals, scope, and non-goals
2. Architecture	High-level and component diagrams (Mermaid)
3. Data Models	Card and GameState types
4. Core Flows	Initialization, interactions, state updates, errors, modal
5. Component Contracts	Props, events
6. Error Handling	Sources, UX, recovery
7. Performance	Re-render strategy, memoization
8. Testing Strategy	Unit, component, integration
9. Risks & Mitigations	Known risks and mitigations
10. Future Work	Enhancements

1. System Overview

The Memory Card Game follows a unidirectional data flow pattern with state lifted to the App component. UI is decomposed into cohesive components: Header, GameBoard, Modal, and an Error Boundary region. Tailwind CSS provides responsive styling and subtle animations.

Goals

- Responsive, accessible gameplay with predictable state transitions. - Clear separation of concerns and small, focused components. - Robust error handling and pleasant win-state UX.

Non-Goals

- No backend persistence or multiplayer features. - No internationalization beyond basic text.

2. Architecture

2.1 High-Level Architecture

```
```mermaid
flowchart TD
 A[User] --> B[React App]
 B --> C[Header]
 B --> D[GameBoard]
 B --> E[Modal]
 B --> F[Error Region]
 B --> G[State (useState)]
 G -->|cards, scores, modal, error| B
 subgraph Styling
 T[Tailwind CSS]
 end
 B --- T
```
```

2.2 Component Architecture

```
```mermaid
flowchart TD
 App --> Header
 App --> GameBoard
 App --> Modal

 subgraph App State
 S1[cards: Card[]]
 S2[currentScore: number]
 S3[bestScore: number]
 S4[error: string]
 S5[isModalOpen: boolean]
 end

 S1 --> GameBoard
 S2 --> Header
 S3 --> Header
 S5 --> Modal
```
```

```

    click GameBoard "#cards" "Renders card grid"
    click Header "#scores" "Displays current & best scores"
    click Modal "#win" "Win dialog"
  ...

```

3. Data Models

3.1 Card

```

```typescript
interface Card {
 id: number; // Unique identifier
 emoji: string; // Card symbol (🀄, 🀅, etc.)
 isFlipped: boolean; // Current flip state
 isMatched: boolean; // Match status
}
```

```

3.2 GameState

```

```typescript
interface GameState {
 cards: Card[]; // 16 cards (8 pairs)
 currentScore: number; // Current attempts or moves
 bestScore: number; // Best achieved
 error: string; // Error message
 isModalOpen: boolean; // Modal visibility
}
```

```

4. Core Flows

4.1 Application Initialization

```

```mermaid
graph TD
 A[App Mount] --> B[useEffect Trigger]
 B --> C[initializeGame()]
 C --> D[Create Card Emojis Array]
 D --> E[Duplicate Array for Pairs]
 E --> F[Shuffle Cards]
 F --> G[Map to Card Objects]
 G --> H[Set Initial State]
 H --> I[Reset Scores & Error]
 I --> J[Game Ready]
```

```

4.2 Card Interaction

```

```mermaid
graph TD
 A[User Clicks Card] --> B[handleCardClick(cardId)]
 B --> C{Valid Click?}
 C -- No --> D[Return Early]
```

```

```

C -- Yes --> E[Find Card by ID]
E --> F[Already Flipped or Matched?]
F -- Yes --> D
F -- No --> G[Count Flipped Cards]
G --> H{Two Already Flipped?}
H -- Yes --> D
H -- No --> I[Flip Card]
I --> J[Update Cards State]
J --> K{Two Cards Now Flipped?}
K -- No --> L[Wait for Second Card]
K -- Yes --> M[Increment Score]
M --> N[Check for Match]
N --> O{Emojis Match?}
O -- Yes --> P[Mark as Matched]
O -- No --> Q[Flip Back After Delay]
P --> R{All Cards Matched?}
R -- Yes --> S[Show Win Modal]
R -- No --> T[Continue Game]
Q --> T
S --> U[Update Best Score]
U --> V[Game Complete]
...

```

4.3 State Update

```

```mermaid
flowchart TD
 A[State Change] --> B[React State Update]
 B --> C[Component Re-render]
 C --> D[Child Components Update]
 D --> E[Props Passed Down]
 E --> F[UI Updates]
 F --> G[User Sees Changes]

 subgraph State Updates
 H[setCards] --> I[Card Grid Re-renders]
 J[setCurrentScore] --> K[Header Updates]
 L[setBestScore] --> K
 M[setError] --> N[Error Message Shows]
 O[setIsModalOpen] --> P[Modal Shows/Hides]
 end
end
...

```

## 4.4 Error Handling

```

```mermaid
flowchart TD
    A[Error Occurs] --> B[Try-Catch Block]
    B --> C[setError(message)]
    C --> D[Error Component Renders]
    D --> E[User Sees Error Message]
    E --> F[Error Auto-Clears on Next Action]

    subgraph Error Sources
        G[Game Initialization] --> H[setError('Failed to initialize game')]
    end

```

```

        I[Card Click Handler] --> J[setError('An error occurred during gameplay')]
        K[State Update Failure] --> L[setError('State update failed')]
    end
    ...

```

4.5 Modal Interaction

```

```mermaid
flowchart TD
 A[Game Won] --> B[setIsModalOpen(true)]
 B --> C[Modal Component Renders]
 C --> D[User Sees Win Message]
 D --> E{User Action}
 E -- Close --> F[handleCloseModal()]
 E -- Restart --> G[handleRestartGame()]
 F --> H[setIsModalOpen(false)]
 G --> I[initializeGame()]
 I --> J[setIsModalOpen(false)]
 H --> K[Modal Hidden]
 J --> K
 K --> L[Game State Reset]
 ...

```

## 5. Component Contracts

### App → Header

Props: { currentScore: number, bestScore: number } – Displays scores.

### App → GameBoard

Props: { cards: Card[], onClick: (cardId: number) => void } – Renders grid; delegates clicks.

### App → Modal

Props: { isOpen: boolean, onClose: () => void, onRestart: () => void } – Win dialog actions.

## 6. Error Handling

Errors surface in a dedicated region; transient errors clear on the next action. For non-recoverable errors, show a friendly message and suggest restarting.

## 7. Performance Considerations

- Lift state and pass stable handlers to minimize re-renders.
- Immutable updates for predictable diffs.
- Memoize heavy derived computations if/when introduced.

## 8. Testing Strategy

- Unit: shuffle, card pairing, match detection.
- Component: GameBoard interactions, Modal visibility.
- Integration: init → play → win → best score update.

## 9. Risks & Mitigations

- Rapid-click race conditions → Temporarily lock input when two cards are face-up. - Accessibility gaps → Focus management, keyboard navigation, ARIA roles. - Mobile layout issues → Responsive grid; test across breakpoints.

## 10. Future Work

- Difficulty levels, timers, and themes. - Persist best score with localStorage. - Animation polish and sound effects.